JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.chrono

Class JapaneseDate

java.lang.Object
    java.time.chrono.JapaneseDate

All Implemented Interfaces:

`Serializable`, `Comparable<ChronoLocalDate>`, `ChronoLocalDate`, `Temporal`, `TemporalAccessor`, `TemporalAdjuster`

```
public final class JapaneseDate
extends Object
implements ChronoLocalDate, Serializable
```

A date in the Japanese Imperial calendar system.

This date operates using the Japanese Imperial calendar. This calendar system is primarily used in Japan.

The Japanese Imperial calendar system is the same as the ISO calendar system apart from the era-based year numbering. The proleptic-year is defined to be equal to the ISO proleptic-year.

Japan introduced the Gregorian calendar starting with Meiji 6. Only Meiji and later eras are supported; dates before Meiji 6, January 1 are not supported.

For example, the Japanese year "Heisei 24" corresponds to ISO year "2012".
Calling `japaneseDate.get(YEAR_OF_ERA)` will return 24.
Calling `japaneseDate.get(YEAR)` will return 2012.
Calling `japaneseDate.get(ERA)` will return 2, corresponding to `JapaneseChronology.ERA_HEISEI`.

This is a value-based class; use of identity-sensitive operations (including reference equality (==), identity hash code, or synchronization) on instances of `JapaneseDate` may have unpredictable results and should be avoided. The `equals` method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Method Summary

Concrete Methods

Modifier and Type	Method	Description
ChronoLocalDateTime<JapaneseDate>	atTime(LocalTime localTime)	Combines this with a time to creat ChronoLocalDa
boolean	equals(Object obj)	Compares this another date, including the chronology.
static JapaneseDate	from(TemporalAccessor temporal)	Obtains a JapaneseDate temporal objec
JapaneseChronology	getChronology()	Gets the chron this date, whic Japanese calen system.
JapaneseEra	getEra()	Gets the era applicable at t
long	getLong(TemporalField field)	Gets the value specified field long.
int	hashCode()	A hash code fo date.
boolean	isSupported(TemporalField field)	Checks if the s field is support

int	**lengthOfMonth**()	Returns the le the month repr by this date.
int	**lengthOfYear**()	Returns the le the year repre by this date, a defined by the calendar syste
JapaneseDate	**minus**(long amountToAdd, **TemporalUnit** unit)	Returns an obj the same type object with the specified perio subtracted.
JapaneseDate	**minus**(**TemporalAmount** amount)	Returns an obj the same type object with an subtracted.
static **JapaneseDate**	**now**()	Obtains the cu JapaneseDate the system clo default time-zo
static **JapaneseDate**	**now**(**Clock** clock)	Obtains the cu JapaneseDate the specified c
static **JapaneseDate**	**now**(**ZoneId** zone)	Obtains the cu JapaneseDate the system clo specified time-
static **JapaneseDate**	**of**(int prolepticYear, int month, int dayOfMonth)	Obtains a JapaneseDate representing a the Japanese c system from th proleptic-year, of-year and da month fields.
static **JapaneseDate**	**of**(**JapaneseEra** era, int yearOfEra, int month, int dayOfMonth)	Obtains a JapaneseDate representing a the Japanese c system from th

		year-of-era, mo year and day-o fields.
JapaneseDate	plus(long amountToAdd, TemporalUnit unit)	Returns an obj the same type object with the specified perio added.
JapaneseDate	plus(TemporalAmount amount)	Returns an obj the same type object with an added.
ValueRange	range(TemporalField field)	Gets the range values for the specified field.
long	toEpochDay()	Converts this d the Epoch Day
String	toString()	Returns a strin representation object.
ChronoPeriod	until(ChronoLocalDate endDate)	Calculates the between this d another date a ChronoPeriod.
long	until(Temporal endExclusive, TemporalUnit unit)	Calculates the of time until an date in terms o specified unit.
JapaneseDate	with(TemporalAdjuster adjuster)	Returns an adj object of the sa type as this ob with the adjust made.
JapaneseDate	with(TemporalField field, long newValue)	Returns an obj the same type object with the specified field

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.time.chrono.ChronoLocalDate

adjustInto, compareTo, format, isAfter, isBefore, isEqual, isLeapYear, isSupported, query, timeLineOrder, toString, until

Methods inherited from interface java.time.temporal.TemporalAccessor

get

Method Detail

now

```
public static JapaneseDate now()
```

Obtains the current `JapaneseDate` from the system clock in the default time-zone.

This will query the `system clock` in the default time-zone to obtain the current date.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

`the current date using the system clock and default time-zone, not null`

now

```
public static JapaneseDate now(ZoneId zone)
```

Obtains the current `JapaneseDate` from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current date. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

`zone - the zone ID to use, not null`

Returns:

`the current date using the system clock, not null`

now

```
public static JapaneseDate now(Clock clock)
```

Obtains the current `JapaneseDate` from the specified clock.

This will query the specified clock to obtain the current date - today. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using dependency injection.

Parameters:

`clock - the clock to use, not null`

Returns:

`the current date, not null`

Throws:

`DateTimeException - if the current date cannot be obtained`

of

```
public static JapaneseDate of(JapaneseEra era,
                              int yearOfEra,
                              int month,
                              int dayOfMonth)
```

Obtains a `JapaneseDate` representing a date in the Japanese calendar system from the era, year-of-era, month-of-year and day-of-month fields.

This returns a `JapaneseDate` with the specified fields. The day must be valid for the year and month, otherwise an exception will be thrown.

The Japanese month and day-of-month are the same as those in the ISO calendar system. They are not reset when the era changes. For example:

```
  6th Jan Showa 64 = ISO 1989-01-06
  7th Jan Showa 64 = ISO 1989-01-07
  8th Jan Heisei 1 = ISO 1989-01-08
  9th Jan Heisei 1 = ISO 1989-01-09
```

Parameters:

era - the Japanese era, not null

yearOfEra - the Japanese year-of-era

month - the Japanese month-of-year, from 1 to 12

dayOfMonth - the Japanese day-of-month, from 1 to 31

Returns:

the date in Japanese calendar system, not null

Throws:

DateTimeException - if the value of any field is out of range, or if the day-of-month is invalid for the month-year, or if the date is not a Japanese era

of

```
public static JapaneseDate of(int prolepticYear,
                              int month,
                              int dayOfMonth)
```

Obtains a JapaneseDate representing a date in the Japanese calendar system from the proleptic-year, month-of-year and day-of-month fields.

This returns a JapaneseDate with the specified fields. The day must be valid for the year and month, otherwise an exception will be thrown.

The Japanese proleptic year, month and day-of-month are the same as those in the ISO calendar system. They are not reset when the era changes.

Parameters:

prolepticYear - the Japanese proleptic-year

month - the Japanese month-of-year, from 1 to 12

dayOfMonth - the Japanese day-of-month, from 1 to 31

Returns:

the date in Japanese calendar system, not null

Throws:

DateTimeException - if the value of any field is out of range, or if the day-of-month is invalid for the month-year

from

```
public static JapaneseDate from(TemporalAccessor temporal)
```

Obtains a JapaneseDate from a temporal object.

This obtains a date in the Japanese calendar system based on the specified temporal. A `TemporalAccessor` represents an arbitrary set of date and time information, which this factory converts to an instance of `JapaneseDate`.

The conversion typically uses the `EPOCH_DAY` field, which is standardized across calendar systems.

This method matches the signature of the functional interface `TemporalQuery` allowing it to be used as a query via method reference, `JapaneseDate::from`.

Specified by:

`from` in interface `ChronoLocalDate`

Parameters:

`temporal - the temporal object to convert, not null`

Returns:

`the date in Japanese calendar system, not null`

Throws:

`DateTimeException - if unable to convert to a JapaneseDate`

See Also:

`Chronology.date(TemporalAccessor)`

getChronology

```
public JapaneseChronology getChronology()
```

Gets the chronology of this date, which is the Japanese calendar system.

The `Chronology` represents the calendar system in use. The era and other fields in `ChronoField` are defined by the chronology.

Specified by:

`getChronology` in interface `ChronoLocalDate`

Returns:

`the Japanese chronology, not null`

getEra

```
public JapaneseEra getEra()
```

Gets the era applicable at this date.

The Japanese calendar system has multiple eras defined by `JapaneseEra`.

Specified by:

getEra in interface ChronoLocalDate

Returns:

the era applicable at this date, not null

lengthOfMonth

```
public int lengthOfMonth()
```

Returns the length of the month represented by this date.

This returns the length of the month in days. Month lengths match those of the ISO calendar system.

Specified by:

lengthOfMonth in interface ChronoLocalDate

Returns:

the length of the month in days

lengthOfYear

```
public int lengthOfYear()
```

Description copied from interface: ChronoLocalDate

Returns the length of the year represented by this date, as defined by the calendar system.

This returns the length of the year in days.

The default implementation uses ChronoLocalDate.isLeapYear() and returns 365 or 366.

Specified by:

lengthOfYear in interface ChronoLocalDate

Returns:

the length of the year in days

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this date can be queried for the specified field. If false, then calling the range and get methods will throw an exception.

If the field is a ChronoField then the query is implemented here. The supported fields are:

- DAY_OF_WEEK
- DAY_OF_MONTH
- DAY_OF_YEAR
- EPOCH_DAY
- MONTH_OF_YEAR
- PROLEPTIC_MONTH
- YEAR_OF_ERA
- YEAR
- ERA

All other ChronoField instances will return false.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.isSupportedBy(TemporalAccessor) passing this as the argument. Whether the field is supported is determined by the field.

Specified by:

isSupported in interface ChronoLocalDate

Specified by:

isSupported in interface TemporalAccessor

Parameters:

field - the field to check, null returns false

Returns:

true if the field is supported on this date, false if not

range

```
public ValueRange range(TemporalField field)
```

Description copied from interface: TemporalAccessor

Gets the range of valid values for the specified field.

All fields can be expressed as a long integer. This method returns an object that describes the valid range for that value. The value of this temporal object is used to enhance the accuracy of the returned range. If the date-time cannot return the range, because the field is unsupported or for some other reason, an exception will be thrown.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

Specified by:

range in interface TemporalAccessor

Parameters:

field - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

getLong

```
public long getLong(TemporalField field)
```

Description copied from interface: TemporalAccessor

Gets the value of the specified field as a long.

This queries the date-time for the value of the specified field. The returned value may be outside the valid range of values for the field. If the date-time cannot return the value, because the field is unsupported or for some other reason, an exception will be thrown.

Specified by:

getLong in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

with

```
public JapaneseDate with(TemporalField field,
                         long newValue)
```

Description copied from interface: ChronoLocalDate

Returns an object of the same type as this object with the specified field altered.

This returns a new object based on this one with the value for the specified field changed. For example, on a LocalDate, this could be used to set the year, month or day-of-month. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then changing the month to February would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

with in interface ChronoLocalDate

Specified by:

with in interface Temporal

Parameters:

field - the field to set in the result, not null

newValue - the new value of the field in the result

Returns:

an object of the same type with the specified field set, not null

with

public JapaneseDate with(TemporalAdjuster adjuster)

Returns an adjusted object of the same type as this object with the adjustment made.

This adjusts this date-time according to the rules of the specified adjuster. A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the date to the last day of the month. A selection of common adjustments is provided in TemporalAdjusters. These include finding the "last day of the month" and "next Wednesday". The adjuster is responsible for handling special cases, such as the varying lengths of month and leap years.

Some example code indicating how and why this method is used:

```
  date = date.with(Month.JULY);        // most key classes implement TemporalAdju
  date = date.with(lastDayOfMonth());  // static import from Adjusters
  date = date.with(next(WEDNESDAY));   // static import from Adjusters and DayOfW
```

Specified by:

with in interface ChronoLocalDate

Specified by:

with in interface Temporal

Parameters:

adjuster - the adjuster to use, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

DateTimeException - if unable to make the adjustment

ArithmeticException - if numeric overflow occurs

plus

```
public JapaneseDate plus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount added.

This adjusts this temporal, adding according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
  date = date.plus(period);                // add a Period instance
  date = date.plus(duration);              // add a Duration instance
  date = date.plus(workingDays(6));        // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Specified by:

`plus` in interface `ChronoLocalDate`

Specified by:

`plus` in interface `Temporal`

Parameters:

`amount` - the amount to add, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if the addition cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
public JapaneseDate minus(TemporalAmount amount)
```

Returns an object of the same type as this object with an amount subtracted.

This adjusts this temporal, subtracting according to the rules of the specified amount. The amount is typically a `Period` but may be any other type implementing the `TemporalAmount` interface, such as `Duration`.

Some example code indicating how and why this method is used:

```
  date = date.minus(period);               // subtract a Period instance
  date = date.minus(duration);             // subtract a Duration instance
  date = date.minus(workingDays(6));       // example user-written workingDays me
```

Note that calling `plus` followed by `minus` is not guaranteed to return the same date-time.

Specified by:

`minus` in interface `ChronoLocalDate`

Specified by:

`minus` in interface `Temporal`

Parameters:

`amount` - the amount to subtract, not null

Returns:

an object of the same type with the specified adjustment made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

plus

```
public JapaneseDate plus(long amountToAdd,
                         TemporalUnit unit)
```

Description copied from interface: `ChronoLocalDate`

Returns an object of the same type as this object with the specified period added.

This method returns a new object based on this one with the specified period added. For example, on a `LocalDate`, this could be used to add a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st January, then adding one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

`plus` in interface `ChronoLocalDate`

Specified by:

`plus` in interface `Temporal`

Parameters:

`amountToAdd - the amount of the specified unit to add, may be negative`

`unit - the unit of the amount to add, not null`

Returns:

`an object of the same type with the specified period added, not null`

minus

```
public JapaneseDate minus(long amountToAdd,
                          TemporalUnit unit)
```

Description copied from interface: `ChronoLocalDate`

Returns an object of the same type as this object with the specified period subtracted.

This method returns a new object based on this one with the specified period subtracted. For example, on a `LocalDate`, this could be used to subtract a number of years, months or days. The returned object will have the same observable type as this object.

In some cases, changing a field is not fully defined. For example, if the target object is a date representing the 31st March, then subtracting one month would be unclear. In cases like this, the field is responsible for resolving the result. Typically it will choose the previous valid date, which would be the last valid day of February in this example.

Specified by:

`minus in interface ChronoLocalDate`

Specified by:

`minus in interface Temporal`

Parameters:

`amountToAdd - the amount of the specified unit to subtract, may be negative`

`unit - the unit of the amount to subtract, not null`

Returns:

`an object of the same type with the specified period subtracted, not null`

atTime

```
public final ChronoLocalDateTime<JapaneseDate> atTime(LocalTime localTime)
```

Description copied from interface: ChronoLocalDate

Combines this date with a time to create a `ChronoLocalDateTime`.

This returns a `ChronoLocalDateTime` formed from this date at the specified time. All possible combinations of date and time are valid.

Specified by:

`atTime` in interface `ChronoLocalDate`

Parameters:

`localTime - the local time to use, not null`

Returns:

`the local date-time formed from this date and the specified time, not null`

until

```
public ChronoPeriod until(ChronoLocalDate endDate)
```

Description copied from interface: ChronoLocalDate

Calculates the period between this date and another date as a `ChronoPeriod`.

This calculates the period between two dates. All supplied chronologies calculate the period using years, months and days, however the `ChronoPeriod` API allows the period to be represented using other units.

The start and end points are `this` and the specified date. The result will be negative if the end is before the start. The negative sign will be the same in each of year, month and day.

The calculation is performed using the chronology of this date. If necessary, the input date will be converted to match.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `ChronoLocalDate`

Parameters:

`endDate - the end date, exclusive, which may be in any chronology, not null`

Returns:

`the period between this date and the end date, not null`

toEpochDay

```
public long toEpochDay()
```

Description copied from interface: ChronoLocalDate

Converts this date to the Epoch Day.

The Epoch Day count is a simple incrementing count of days where day 0 is 1970-01-01 (ISO). This definition is the same for all chronologies, enabling conversion.

This default implementation queries the EPOCH_DAY field.

Specified by:

toEpochDay in interface ChronoLocalDate

Returns:

the Epoch Day equivalent to this date

equals

```
public boolean equals(Object obj)
```

Compares this date to another date, including the chronology.

Compares this JapaneseDate with another ensuring that the date is the same.

Only objects of type JapaneseDate are compared, other types return false. To compare the dates of two TemporalAccessor instances, including dates in two different chronologies, use ChronoField.EPOCH_DAY as a comparator.

Specified by:

equals in interface ChronoLocalDate

Parameters:

obj - the object to check, null returns false

Returns:

true if this is equal to the other date

See Also:

Object.hashCode(), HashMap

hashCode

```
public int hashCode()
```

A hash code for this date.

Specified by:

hashCode in interface ChronoLocalDate

Returns:

a suitable hash code based only on the Chronology and the date

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Description copied from interface: `ChronoLocalDate`

Calculates the amount of time until another date in terms of the specified unit.

This calculates the amount of time between two `ChronoLocalDate` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified date. The result will be negative if the end is before the start. The `Temporal` passed to this method is converted to a `ChronoLocalDate` using `Chronology.date(TemporalAccessor)`. The calculation returns a whole number, representing the number of complete units between the two dates. For example, the amount in days between two dates can be calculated using `startDate.until(endDate, DAYS)`.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, MONTHS);
amount = MONTHS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `DAYS`, `WEEKS`, `MONTHS`, `YEARS`, `DECADES`, `CENTURIES`, `MILLENNIA` and `ERAS` should be supported by all implementations. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `ChronoLocalDate`

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive - the end date, exclusive, which is converted to a ChronoLocalDate in the same chronology, not null`

`unit - the unit to measure the amount in, not null`

Returns:

`the amount of time between this date and the end date`

toString

```
public String toString()
```

Description copied from class: `Object`

Returns a string representation of the object. In general, the `toString` method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read. It is recommended that all subclasses override this method.

The `toString` method for class `Object` returns a string consisting of the name of the class of which the object is an instance, the at-sign character `@', and the unsigned hexadecimal representation of the hash code of the object. In other words, this method returns a string equal to the value of:

```
getClass().getName() + '@' + Integer.toHexString(hashCode())
```

Specified by:

`toString` in interface `ChronoLocalDate`

Overrides:

`toString` in class `Object`

Returns:

`a string representation of the object.`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.